

LABORATORY WORK 2

Normalization

Task 1. Identifying Normal Form Violations

The Booking_receipt table contains information about bookings, passengers, flights, airports, airlines, seats, and ticket prices.

1NF Violation:

The seat_numbers column stores multiple values in one cell, such as "12A, 12B". This violates 1NF because each field should contain a single atomic value.

2NF Violation:

After converting the table to 1NF, we can use (booking_id, seat_number) as a composite key, assuming each seat appears only once within a booking. Some attributes, such as flight_number and ticket_price, depend only on booking_id rather than the entire composite key. This creates a partial dependency.

3NF Violation:

The table contains transitive dependencies. For example, booking_id determines flight_number, and flight_number determines airline_name. Similarly, departure_airport_name determines departure_city, assuming airport names are unique. These attributes should be stored in separate tables to reduce redundancy.

Conclusion:

The original Booking_receipt table is not in 1NF because seat_numbers contain multiple values. After fixing this problem, additional normalization is needed to remove partial and transitive dependencies.

Task 2. Step-by-step Normalization

Step 1. First Normal Form (1NF)

The *seat_numbers* column contains multiple values in one cell. To achieve 1NF, each seat number is stored in a separate row.

Booking_receipt_1NF

booking_id	passenger_passport_number	seat number
101	P1001	12A
101	P1002	12B
102	P1003	15C

Primary Key (PK): (*booking_id*, *seat_number*)

All remaining attributes from the original `Booking_receipt` table are retained.

Result: Each cell contains one atomic value.

Step 2. Second Normal Form (2NF)

To achieve 2NF, we remove partial dependencies by separating booking information from passenger and seat information.

Table 1: Booking_2NF

Attribute	Key
booking_id	PK
flight_number	-
departure_airport_name	-
departure_city	-
arrival_airport_name	-
arrival_city	-
airline_name	-

Table 2: Booking_seat_2NF

Attribute	key
booking_id	PK, FK
seat_number	PK
passenger_passport_number	-
passenger_full_name	-
ticket_price	-

Primary Key: (booking_id, seat_number) Foreign Key: booking_id → Booking_2NF(booking_id)

Result: Partial dependencies are removed.

Step 3: Third Normal Form (3NF)

To achieve 3NF, we remove transitive dependencies and separate booking, passenger, flight, airport, airline, and seat information into different tables.

Table 1: Booking_3NF

Attribute	Key
booking_id	PK
flight_number	FK

Table 2: Passenger_3NF

Attribute	Key
passenger_passport_number	PK
passenger_full_name	-

Table 3: Booking_seat_3NF

Attribute	Key
booking_id	PK, FK
seat_number	PK
passenger_passport_number	FK
ticket_price	-

Table 4: Flight_3NF

Attribute	Key
flight_number	PK
departure_airport_name	FK
arrival_airport_name	FK
airline_name	FK

Table 5: Airport_3NF

Attribute	Key
airport_name	PK
city	-

Table 6: Airline_3NF

Attribute	Key
airport_name	PK

Composite Primary Key: *Booking_seat_3NF* (*booking_id*,
seat_number)

Conclusion:

The Booking_receipt table has been normalized into six separate tables. Each table stores information about one main entity or relationship.

This structure reduces data redundancy, removes partial and transitive dependencies, and improves data consistency.

The design assumes that flight numbers and airport names uniquely identify flights and airports, respectively, and that each seat is assigned to at most one passenger within a booking.

Task 3. Functional Dependencies

The following functional dependencies were used to normalize the Booking_receipt table.

1. Dependencies used for 1NF

The seat_numbers column contains multiple values. It was separated into individual seat_number values to ensure atomicity.

2. Dependencies used for 2NF

booking_id → flight_number

(booking_id, seat_number) → passenger_passport_number, ticket_price

These dependencies show that booking information depends only on booking_id rather than the entire composite key (booking_id, seat_number).

3. Dependencies used for 3NF

passenger_passport_number → passenger_full_name

flight_number → departure_airport_name, arrival_airport_name, airline_name

departure_airport_name → departure_city

arrival_airport_name → arrival_city

These dependencies justify separating passengers, flights, airports, airlines, bookings, and individual seats into different tables.

Conclusion:

Functional dependencies help identify partial and transitive dependencies. By removing these dependencies, the database becomes more organized and reduces unnecessary duplication of information.

Task 4. Checking the Flights Table for 3NF

1. Primary Key

The primary key of the Flights table is `flight_id`. It uniquely identifies each flight.

2. Functional Dependencies

`flight_id` → `sch_departure_time`, `sch_arrival_time`, `departing_airport_id`, `arriving_airport_id`, `departing_gate`, `arriving_gate`, `airline_id`, `act_departure_time`, `act_arrival_time`, `created_at`, `updated_at`.

All non-key attributes depend on `flight_id`.

3. Transitive Dependencies

The Flights table contains `departing_airport_id`, `arriving_airport_id`, and `airline_id` as foreign keys.

Airport details, such as `airport_name` and `city`, are stored in the Airport table. Airline details, such as `airline_name`, are stored in the Airline table.

Therefore, these descriptive attributes do not create transitive dependencies within Flights.

4. Conclusion

The Flights table satisfies 3NF under the stated functional dependencies because all attributes are atomic, the primary key consists of one column, and no non-key attribute transitively determines another non-key attribute.

The screenshot shows a PostgreSQL client interface with a query window and a results window. The query window contains the following SQL query:

```

SELECT
    column_name,
    data_type,
    is_nullable
FROM information_schema.columns
WHERE table_name = 'flights'
ORDER BY ordinal_position;

```

The results window displays the output of the query, showing the structure of the 'flights' table. The results are as follows:

	column_name	data_type	is_nullable
1	flight_id	integer	NO
2	sch_departure_time	timestamp without time zone	NO
3	sch_arrival_time	timestamp without time zone	NO
4	departing_airport_id	integer	NO
5	arriving_airport_id	integer	NO
6	departing_gate	text	NO
7	arriving_gate	character varying	NO
8	airline_id	integer	NO
9	act_departure_time	timestamp without time zone	NO
10	act_arrival_time	timestamp without time zone	NO
11	created_at	timestamp without time zone	NO
12	updated_at	timestamp without time zone	NO

Figure 1. Verification of the Flights table structure in PostgreSQL.

Task 5. Implementation of 3NF Tables.

```
Query Query History
306
307 CREATE TABLE receipt_passenger (
308     passenger_passport_number VARCHAR(20) PRIMARY KEY,
309     passenger_full_name VARCHAR(100) NOT NULL
310 );
311
312 CREATE TABLE receipt_airport (
313     airport_name VARCHAR(100) PRIMARY KEY,
314     city VARCHAR(100) NOT NULL
315 );
316
317 CREATE TABLE receipt_airline (
318     airline_name VARCHAR(100) PRIMARY KEY
319 );
320
321 CREATE TABLE receipt_flight (
322     flight_number VARCHAR(20) PRIMARY KEY,
323     departure_airport_name VARCHAR(100) NOT NULL,
324     arrival_airport_name VARCHAR(100) NOT NULL,
325     airline_name VARCHAR(100) NOT NULL,
326
327     FOREIGN KEY (departure_airport_name)
328         REFERENCES receipt_airport(airport_name),
329
330     FOREIGN KEY (arrival_airport_name)
331         REFERENCES receipt_airport(airport_name),
332
333     FOREIGN KEY (airline_name)
334         REFERENCES receipt_airline(airline_name)
335 );
336
```

```
Query History
359
360 INSERT INTO receipt_passenger
361 (passenger_passport_number, passenger_full_name)
362 VALUES
363 ('P1001', 'Aigerim Sadykova'),
364 ('P1002', 'Dias Omarov'),
365 ('P1003', 'Dana Kim'),
366 ('P1004', 'Arman Nurgali'),
367 ('P1005', 'Madina Aliyeva');
368
369 INSERT INTO receipt_airport
370 (airport_name, city)
371 VALUES
372 ('Almaty International Airport', 'Almaty'),
373 ('Astana International Airport', 'Astana'),
374 ('Istanbul Airport', 'Istanbul');
375
376 INSERT INTO receipt_airline
377 (airline_name)
378 VALUES
379 ('Air Astana'),
380 ('Turkish Airlines');
381
382 INSERT INTO receipt_flight
383 (flight_number, departure_airport_name, arrival_airport_name, airline_name)
384 VALUES
385 ('KC101', 'Almaty International Airport', 'Astana International Airport', 'Air
386 ('TK351', 'Almaty International Airport', 'Istanbul Airport', 'Turkish Airlines
387
388 INSERT INTO receipt_booking
389 (booking_id, flight_number)
```

Result:

Dashboard × Properties × SQL × Statistics × Dependencies × Dependents × Processes × airport_lab2/postgres@PostgreSQL 18

Query History

```
401 (103, '18A', 'P1004', 48000.00);
402 (103, '18B', 'P1005', 48000.00);
403
404 ALTER TABLE receipt_booking_seat
405 ALTER COLUMN ticket_price TYPE DECIMAL(10,2);
406
407 SELECT *
408 FROM receipt_booking_seat
409 ORDER BY booking_id, seat_number;
```

Data Output Messages Notifications

Showing rows: 1 to 5

	booking_id [PK] integer	seat_number [PK] character varying (10)	passenger_passport_number character varying (20)	ticket_price numeric (10,2)
1	101	12A	P1001	45000.00
2	101	12B	P1002	45000.00
3	102	15C	P1003	120000.00
4	103	18A	P1004	48000.00
5	103	18B	P1005	48000.00

DDL

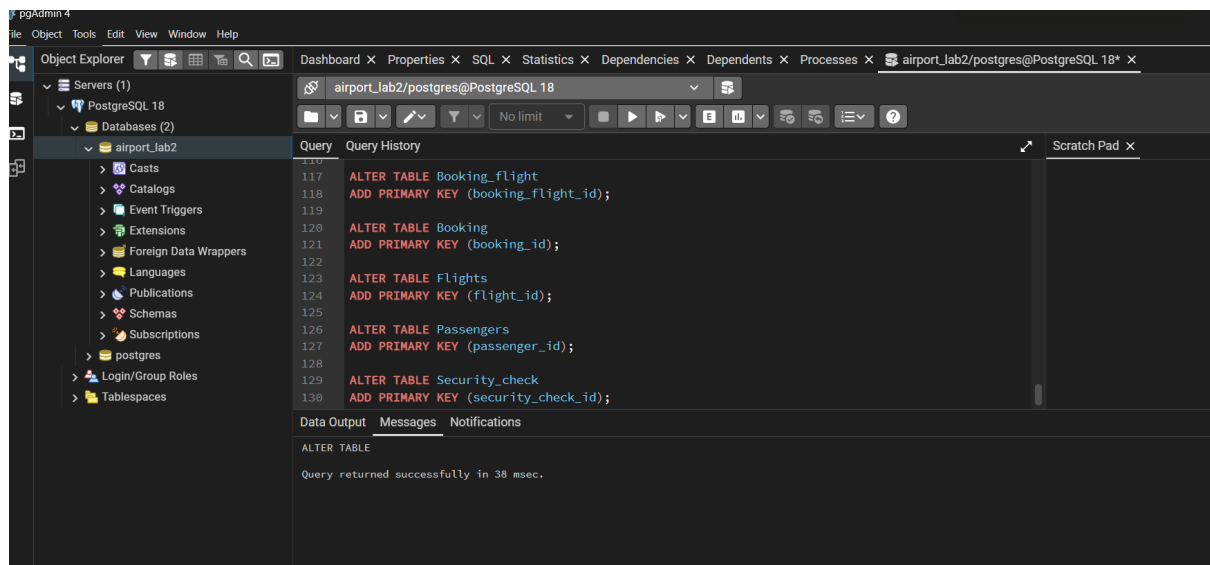
1.CREATE TABLE

The screenshot shows the pgAdmin 4 interface with the following components:

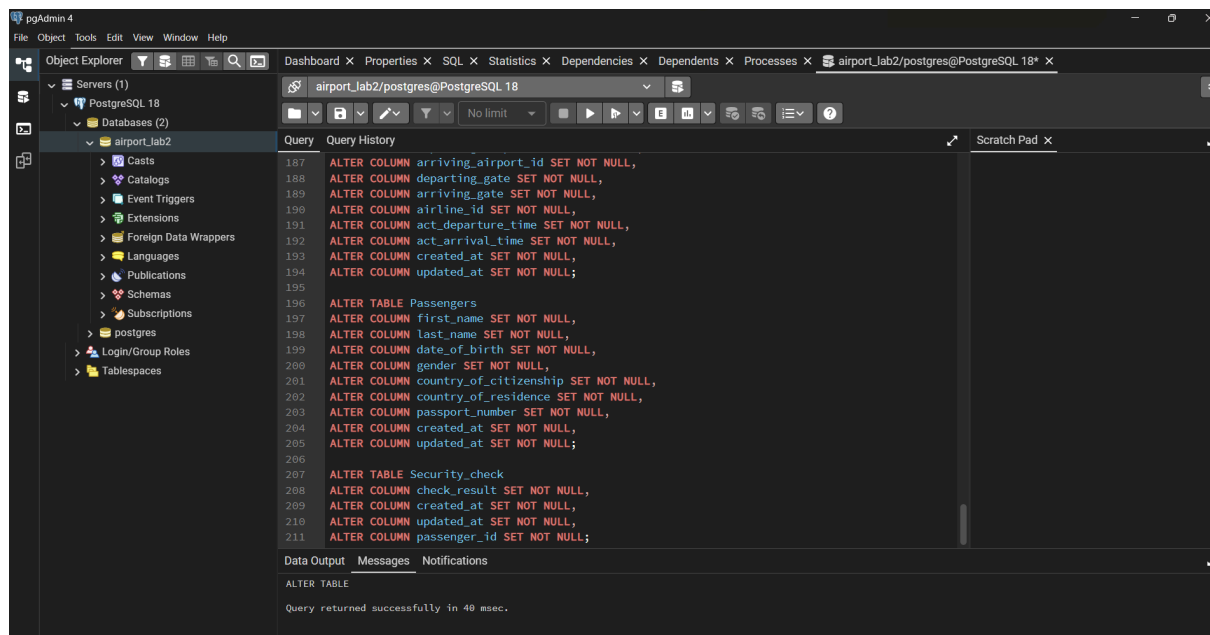
- Left Panel:** A tree view showing the database structure. It includes 'Servers (1)' with 'PostgreSQL 18', which contains 'Databases (2)'. The 'airport_lab2' database is selected and expanded, showing various objects like 'Casts', 'Catalogs', 'Event Triggers', 'Extensions', 'Foreign Data W', 'Languages', 'Publications', 'Schemas', 'Subscriptions', 'postgres', 'Login/Group Roles', and 'Tablespaces'.
- Top Panel:** A toolbar with icons for file operations (New, Open, Save, Print), editing (Undo, Redo), and execution (Execute, Stop, Refresh). A dropdown menu shows 'airport_lab2/postgres@PostgreSQL 18'.
- Query Editor:** A text area containing the SQL command:

```
1 CREATE TABLE Airline_info (  
2     airline_id INT,  
3     airline_code VARCHAR(30),  
4     airline_name VARCHAR(50),  
5     airline_country VARCHAR(50),  
6     created_at TIMESTAMP,  
7     updated_at TIMESTAMP,  
8     info VARCHAR(50)  
9 );
```
- Data Output Panel:** A section with tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the message: 'CREATE TABLE' and 'Query returned successfully in 45 msec.'
- Status Bar:** At the bottom, it displays 'Total rows: Query complete 00:00:00.045', 'CRLF', and 'Ln 9, Col 3'.

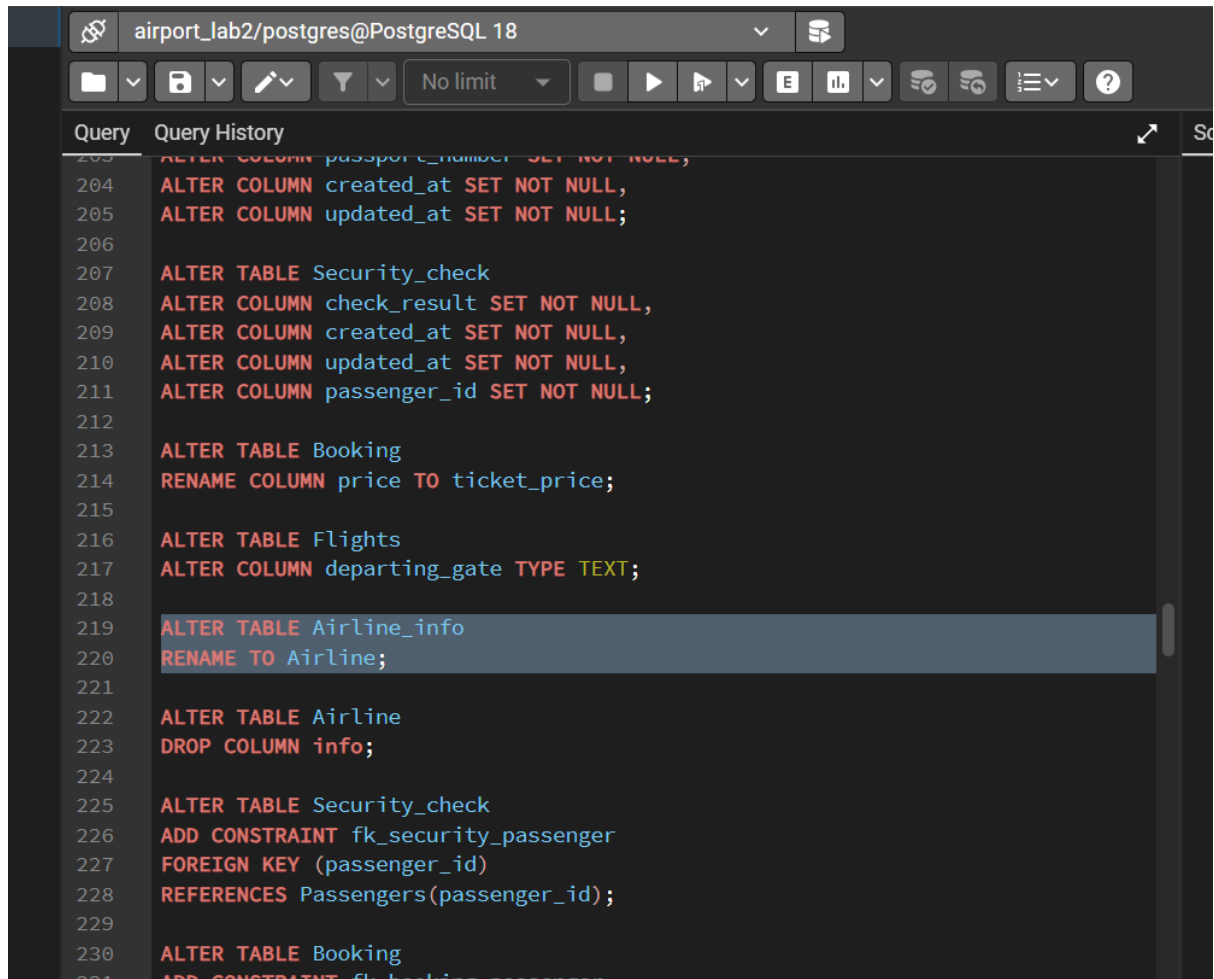
2.PRIMARY KEY



4.NOT NULL

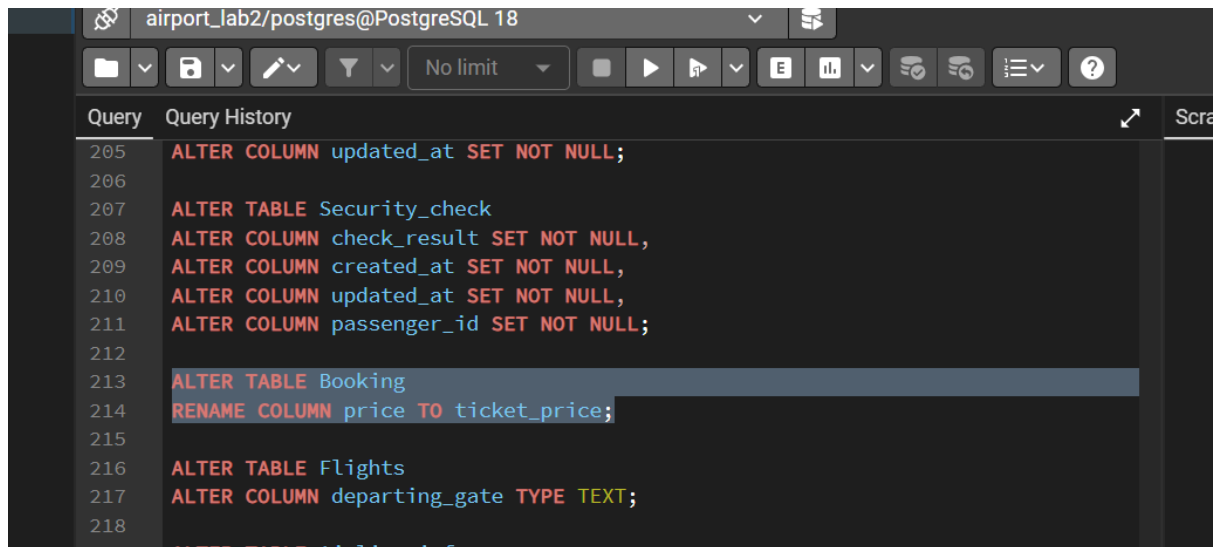


5. Rename airline_info table to airline



```
203 ALTER COLUMN passport_number SET NOT NULL;
204 ALTER COLUMN created_at SET NOT NULL;
205 ALTER COLUMN updated_at SET NOT NULL;
206
207 ALTER TABLE Security_check
208 ALTER COLUMN check_result SET NOT NULL,
209 ALTER COLUMN created_at SET NOT NULL,
210 ALTER COLUMN updated_at SET NOT NULL,
211 ALTER COLUMN passenger_id SET NOT NULL;
212
213 ALTER TABLE Booking
214 RENAME COLUMN price TO ticket_price;
215
216 ALTER TABLE Flights
217 ALTER COLUMN departing_gate TYPE TEXT;
218
219 ALTER TABLE Airline_info
220 RENAME TO Airline;
221
222 ALTER TABLE Airline
223 DROP COLUMN info;
224
225 ALTER TABLE Security_check
226 ADD CONSTRAINT fk_security_passenger
227 FOREIGN KEY (passenger_id)
228 REFERENCES Passengers(passenger_id);
229
230 ALTER TABLE Booking
231 ADD CONSTRAINT fk_booking_passenger
```

6. Rename column price to ticket_price in booking table



```
205 ALTER COLUMN updated_at SET NOT NULL;
206
207 ALTER TABLE Security_check
208 ALTER COLUMN check_result SET NOT NULL,
209 ALTER COLUMN created_at SET NOT NULL,
210 ALTER COLUMN updated_at SET NOT NULL,
211 ALTER COLUMN passenger_id SET NOT NULL;
212
213 ALTER TABLE Booking
214 RENAME COLUMN price TO ticket_price;
215
216 ALTER TABLE Flights
217 ALTER COLUMN departing_gate TYPE TEXT;
218
219 ALTER TABLE Airline_info
```

7. Change data type of departing_gate from varchar(50) to text

```
208 ALTER COLUMN check_result SET NOT NULL,
209 ALTER COLUMN created_at SET NOT NULL,
210 ALTER COLUMN updated_at SET NOT NULL,
211 ALTER COLUMN passenger_id SET NOT NULL;
212
213 ALTER TABLE Booking
214 RENAME COLUMN price TO ticket_price;
215
216 ALTER TABLE Flights
217 ALTER COLUMN departing_gate TYPE TEXT;
218
219 ALTER TABLE Airline_info
220 RENAME TO Airline;
221
222 ALTER TABLE Airline
```

8. Drop the column info(varchar(50)) from the airline table.

```
218
219 ALTER TABLE Airline_info
220 RENAME TO Airline;
221
222 ALTER TABLE Airline
223 DROP COLUMN info;
224
225 ALTER TABLE Security_check
226 ADD CONSTRAINT fk_security_passenger
227 FOREIGN KEY (passenger_id)
```

9.

The screenshot shows the pgAdmin 4 interface. On the left, the 'Object Explorer' pane displays the database structure for 'airport_lab2', including 'Casts', 'Catalogs', 'Event Triggers', 'Extensions', 'Foreign Data Wrappers', 'Languages', 'Publications', 'Schemas', 'Subscriptions', 'postgres', 'Login/Group Roles', and 'Tablespaces'. The main pane shows a SQL query being executed. The query is as follows:

```
262 FOREIGN KEY (flight_id)
263 REFERENCES Flights(flight_id);
264
265 ALTER TABLE Flights
266 ADD CONSTRAINT fk_flights_departure
267 FOREIGN KEY (departing_airport_id)
268 REFERENCES Airport(airport_id);
269
270 ALTER TABLE Flights
271 ADD CONSTRAINT fk_flights_arrival
272 FOREIGN KEY (arriving_airport_id)
273 REFERENCES Airport(airport_id);
274
275 ALTER TABLE Flights
276 ADD CONSTRAINT fk_flights_airline
277 FOREIGN KEY (airline_id)
278 REFERENCES Airline(airline_id);
```

The 'Data Output' pane at the bottom shows the result of the query: 'ALTER TABLE' and 'Query returned successfully in 37 msec.'

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer Servers (1) PostgreSQL 18 Databases (2) airport_lab2

airpot_lab2/postgres@PostgreSQL 18

Query

```
279
280 SELECT
281     tc.table_name,
282     tc.constraint_name,
283     kcu.column_name,
284     ccu.table_name AS referenced_table,
285     ccu.column_name AS referenced_column
286 FROM information_schema.table_constraints AS tc
287 JOIN information_schema.key_column_usage AS kcu
288     ON tc.constraint_name = kcu.constraint_name
289     AND tc.table_schema = kcu.table_schema
290 JOIN information_schema.constraint_column_usage AS ccu
291     ON tc.constraint_name = ccu.constraint_name
292     AND tc.table_schema = ccu.table_schema
293 WHERE tc.constraint_type = 'FOREIGN KEY'
294     AND tc.table_schema = 'public'
295 ORDER BY tc.table_name, tc.constraint_name;
```

Data Output Messages Notifications

Showing rows: 1 to 11 Page No: 1 of 1

table_name	constraint_name	column_name	referenced_table	referenced_column_name
baggage	fk_baggage_booking	booking_id	booking	booking_id
baggage_ch...	fk_baggage_check_booking	booking_id	booking	booking_id
baggage_ch...	fk_baggage_check_passen...	passenger_id	passengers	passenger_id
boarding_pa...	fk_boarding_booking	booking_id	booking	booking_id
booking	fk_booking_passenger	passenger_id	passengers	passenger_id
booking_flight	fk_booking_flight_booking	booking_id	booking	booking_id
booking_flight	fk_booking_flight_flight	flight_id	flights	flight_id

Total rows: 11 Query complete 00:00:00.060 CRLF Ln 295, C

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer Servers (1) PostgreSQL 18 Databases (2) airport_lab2

airpot_lab2/postgres@PostgreSQL 18

Query

```
279
280 SELECT
281     tc.table_name,
282     tc.constraint_name,
283     kcu.column_name,
284     ccu.table_name AS referenced_table,
285     ccu.column_name AS referenced_column
286 FROM information_schema.table_constraints AS tc
287 JOIN information_schema.key_column_usage AS kcu
288     ON tc.constraint_name = kcu.constraint_name
289     AND tc.table_schema = kcu.table_schema
290 JOIN information_schema.constraint_column_usage AS ccu
291     ON tc.constraint_name = ccu.constraint_name
292     AND tc.table_schema = ccu.table_schema
293 WHERE tc.constraint_type = 'FOREIGN KEY'
294     AND tc.table_schema = 'public'
295 ORDER BY tc.table_name, tc.constraint_name;
```

Data Output Messages Notifications

Showing rows: 1 to 11 Page No: 1 of 1

table_name	constraint_name	column_name	referenced_table	referenced_column_name
baggage	fk_baggage_booking	booking_id	booking	booking_id
baggage_ch...	fk_baggage_check_booking	booking_id	booking	booking_id
baggage_ch...	fk_baggage_check_passen...	passenger_id	passengers	passenger_id
boarding_pa...	fk_boarding_booking	booking_id	booking	booking_id
booking	fk_booking_passenger	passenger_id	passengers	passenger_id
booking_flight	fk_booking_flight_booking	booking_id	booking	booking_id
booking_flight	fk_booking_flight_flight	flight_id	flights	flight_id
flights	fk_flights_airline	airline_id	airline	airline_id
flights	fk_flights_arrival	arriving_airport_id	airport	airport_id
flights	fk_flights_departure	departing_airport...	airport	airport_id
security_che...	fk_security_passenger	passenger_id	passengers	passenger_id